

# Regulatory Action Composition

Jinwook Kim (Jay)

July 30, 2026

## Abstract

We mechanize sequential regulatory-action composition over a concrete five-state, seven-label reference machine in Isabelle/HOL. The theory separates applied transitions, legal rejection, and operational failure; classifies all 21 unordered pairs of distinct transition labels into 12 commuting and 9 noncommuting pairs with checked witnesses; proves terminal absorption and restrictive-state provenance; specifies typed legal effects and state-sensitive transfer gates; records trace and frame properties for commands and completed atomic synchronization steps; and enumerates exactly 60 distinct reachable state-transformation vectors. These are model-level results for the declared finite machine. They do not establish legal truth, authorization correctness, concurrent execution, distributed rollback, network liveness, or model-to-code refinement.

## Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Overview</b>                                  | <b>2</b>  |
| 1.1      | Checked result groups . . . . .                  | 2         |
| 1.2      | Assurance boundary . . . . .                     | 3         |
| <b>2</b> | <b>State Effects and Legal Outcomes</b>          | <b>3</b>  |
| <b>3</b> | <b>Complete Pair Classification</b>              | <b>4</b>  |
| <b>4</b> | <b>Terminal Absorption and State Provenance</b>  | <b>7</b>  |
| <b>5</b> | <b>Typed Legal-Effect Descriptors</b>            | <b>10</b> |
| <b>6</b> | <b>Ordinary and Enforcement Transfer Gates</b>   | <b>12</b> |
| <b>7</b> | <b>Reference Roundtrips and Information Loss</b> | <b>14</b> |
| <b>8</b> | <b>Commands, Trace, and Frame Properties</b>     | <b>14</b> |
| <b>9</b> | <b>Atomic Synchronization Queues</b>             | <b>17</b> |

|                                              |           |
|----------------------------------------------|-----------|
| <b>10 Finite Transformation Normal Forms</b> | <b>21</b> |
| <b>11 Conclusion</b>                         | <b>26</b> |

## 1 Overview

Regulatory commands do not in general commute. Applying two otherwise valid labels in opposite orders may produce different final states because the first transition changes whether the second transition is defined. This artifact makes that order sensitivity explicit and executable at the model level, while preserving a distinction between an undefined legal transition and an operational failure outside the state machine.

The session is a standalone artifact built on the published `Cross_Domain_State_Preservation` session. It imports only the concrete regulatory instance from that session; it does not import or extend the authenticated-data-structure functor layer. The shared base supplies the five states, seven transition labels, transition function, and atomic global state abstraction [1]. The regulatory interpretation is motivated by the protocol model described in the related RCP paper [2].

### 1.1 Checked result groups

**Outcome semantics.** Applied, rejected, and operational-failure outcomes remain distinct, with rejected and failed commands leaving the modeled state unchanged.

**Pair classification.** The complete set of 21 unordered pairs of distinct labels is partitioned into 12 commuting and 9 noncommuting pairs; every noncommuting pair has a concrete state witness.

**Terminality and provenance.** Confiscation is absorbing for arbitrary action words, while the reachable restrictive states carry checked provenance constraints.

**Legal-effect and transfer layer.** Six typed legal-effect descriptors cover state transitions and the separate enforcement-transfer path. Ordinary transfers use both the regulatory-state gate and a current-policy condition.

**Trace and synchronization.** Command queues preserve order and input identity, record execution outcome, leave unrelated assets unchanged, and preserve validity after each completed atomic synchronization step.

**Finite normal forms.** Exactly 60 distinct state-transformation vectors are reachable; every action word evaluates to one of them and the set is closed under every transition label.

## 1.2 Assurance boundary

The artifact is a finite reference semantics, not a legal opinion or an implementation proof. In particular, the current-policy input for restricted transfers is abstract. Asset lookup, authorization, replay protection, concurrent interleavings, partial propagation, timeout, rollback, bytecode behavior, and deployment security remain external or refinement obligations. The atomic queue theorems speak only about completed model steps.

```
theory Regulatory-Action-Composition
imports Cross-Domain-State-Preservation.Regulatory-Instance
begin
```

## 2 State Effects and Legal Outcomes

```
datatype legal-rejection-reason = Undefined-Transition
```

```
datatype operational-failure-reason =
  Missing-Asset
  | Lock-Unavailable
  | External-Failure
```

```
datatype command-outcome =
  Applied reg-state
  | Rejected legal-rejection-reason
  | Operational-Failure operational-failure-reason
```

```
fun state-after :: reg-state  $\Rightarrow$  command-outcome  $\Rightarrow$  reg-state where
  state-after s (Applied s') = s'
  | state-after s (Rejected r) = s
  | state-after s (Operational-Failure r) = s
```

```
definition legal-outcome :: reg-state  $\Rightarrow$  reg-action  $\Rightarrow$  command-outcome where
  legal-outcome s a =
    (case reg-transition s a of
      Some s'  $\Rightarrow$  Applied s'
    | None  $\Rightarrow$  Rejected Undefined-Transition)
```

```
definition state-effect :: reg-state  $\Rightarrow$  reg-action  $\Rightarrow$  reg-state where
  state-effect s a = state-after s (legal-outcome s a)
```

```
theorem action-state-idempotent:
  state-effect (state-effect s a) a = state-effect s a
```

**by** (*cases s*; *cases a*; *simp add: state-effect-def legal-outcome-def*)

**lemma** *successful-repeat-is-rejected*:  
**assumes** *reg-transition s a = Some s'*  
**shows** *legal-outcome s a = Applied s' ∧*  
*legal-outcome s' a = Rejected Undefined-Transition*  
**using** *assms*  
**by** (*cases s*; *cases a*; *simp add: legal-outcome-def*)

**lemma** *rejected-repeat-stays-rejected*:  
**assumes** *reg-transition s a = None*  
**shows** *legal-outcome s a = Rejected Undefined-Transition ∧*  
*legal-outcome (state-effect s a) a = Rejected Undefined-Transition*  
**using** *assms*  
**by** (*cases s*; *cases a*; *simp add: state-effect-def legal-outcome-def*)

**theorem** *concrete-rejected-repeat-stays-rejected*:  
*legal-outcome FROZEN FREEZE = Rejected Undefined-Transition ∧*  
*legal-outcome (state-effect FROZEN FREEZE) FREEZE =*  
*Rejected Undefined-Transition*  
**by** (*simp add: state-effect-def legal-outcome-def*)

**fun** *run-word* :: *reg-action list ⇒ reg-state ⇒ reg-state* **where**  
*run-word [] s = s*  
| *run-word (a # as) s = run-word as (state-effect s a)*

**fun** *action-outcome-queue* ::  
*reg-action list ⇒ reg-state ⇒ reg-state × command-outcome list*  
**where**  
*action-outcome-queue [] s = (s, [])*  
| *action-outcome-queue (a # as) s =*  
*(let out = legal-outcome s a;*  
*rest = action-outcome-queue as (state-after s out)*  
*in (fst rest, out # snd rest))*

**lemma** *action-outcome-queue-state*:  
*fst (action-outcome-queue as s) = run-word as s*  
**by** (*induction as arbitrary: s*) (*simp-all add: Let-def state-effect-def*)

**lemma** *run-word-append*:  
*run-word (xs @ ys) s = run-word ys (run-word xs s)*  
**by** (*induction xs arbitrary: s*) *simp-all*

### 3 Complete Pair Classification

**definition** *all-reg-states* :: *reg-state list* **where**  
*all-reg-states = [ACTIVE, FROZEN, SEIZED, CONFISCATED, RE-*  
*STRICTED]*

**definition** *all-reg-actions* :: *reg-action list* **where**

*all-reg-actions* =  
 [FREEZE, SEIZE, CONFISCATE, RESTRICT, UNFREEZE, UNRESTRICT,  
 RELEASE]

**definition** *commutes* :: *reg-action*  $\Rightarrow$  *reg-action*  $\Rightarrow$  *bool* **where**

*commutes* *a b*  $\longleftrightarrow$   
 ( $\forall s$ . *state-effect* (*state-effect* *s a*) *b* =  
*state-effect* (*state-effect* *s b*) *a*)

**definition** *pair-commutes* :: *reg-action*  $\times$  *reg-action*  $\Rightarrow$  *bool* **where**

*pair-commutes* *p* =  
 (case *p* of (*a*, *b*)  $\Rightarrow$   
*list-all* ( $\lambda s$ . *state-effect* (*state-effect* *s a*) *b* =  
*state-effect* (*state-effect* *s b*) *a*) *all-reg-states*)

**definition** *all-unordered-pairs* :: (*reg-action*  $\times$  *reg-action*) *list* **where**

*all-unordered-pairs* =  
 [(FREEZE, SEIZE), (FREEZE, CONFISCATE), (FREEZE, RESTRICT),  
 (FREEZE, UNFREEZE), (FREEZE, UNRESTRICT), (FREEZE, RE-  
 LEASE),  
 (SEIZE, CONFISCATE), (SEIZE, RESTRICT), (SEIZE, UNFREEZE),  
 (SEIZE, UNRESTRICT), (SEIZE, RELEASE),  
 (CONFISCATE, RESTRICT), (CONFISCATE, UNFREEZE),  
 (CONFISCATE, UNRESTRICT), (CONFISCATE, RELEASE),  
 (RESTRICT, UNFREEZE), (RESTRICT, UNRESTRICT), (RESTRICT,  
 RELEASE),  
 (UNFREEZE, UNRESTRICT), (UNFREEZE, RELEASE),  
 (UNRESTRICT, RELEASE)]

**theorem** *all-unordered-pairs-distinct*:

*distinct all-unordered-pairs*  
**by** (*simp add: all-unordered-pairs-def*)

**theorem** *all-unordered-pairs-irreflexive*:

(*a*, *a*)  $\notin$  *set all-unordered-pairs*  
**by** (*cases a*) (*simp-all add: all-unordered-pairs-def*)

**theorem** *all-unordered-pairs-complete*:

**assumes** *a*  $\neq$  *b*  
**shows** (*a*, *b*)  $\in$  *set all-unordered-pairs*  $\vee$   
 (*b*, *a*)  $\in$  *set all-unordered-pairs*  
**using** *assms*  
**by** (*cases a*; *cases b*; *simp add: all-unordered-pairs-def*)

**definition** *commuting-pairs* :: (*reg-action*  $\times$  *reg-action*) *list* **where**

*commuting-pairs* =  
 [(FREEZE, CONFISCATE), (FREEZE, RESTRICT), (FREEZE, UNRE-  
 STRICT),

$(SEIZE, CONFISCATE), (SEIZE, UNFREEZE),$   
 $(CONFISCATE, RESTRICT), (CONFISCATE, UNFREEZE),$   
 $(CONFISCATE, UNRESTRICT), (CONFISCATE, RELEASE),$   
 $(UNFREEZE, UNRESTRICT), (UNFREEZE, RELEASE),$   
 $(UNRESTRICT, RELEASE)]$

**definition** *noncommuting-pairs* :: (reg-action  $\times$  reg-action) list **where**  
*noncommuting-pairs* =  
 $[(FREEZE, SEIZE), (FREEZE, UNFREEZE), (FREEZE, RELEASE),$   
 $(SEIZE, RESTRICT), (SEIZE, UNRESTRICT), (SEIZE, RELEASE),$   
 $(RESTRICT, UNFREEZE), (RESTRICT, UNRESTRICT), (RESTRICT,$   
 $RELEASE)]$

**definition** *noncommuting-witnesses* ::  
 $((reg-action \times reg-action) \times reg-state)$  list  
**where**  
*noncommuting-witnesses* =  
 $[((FREEZE, SEIZE), RESTRICTED),$   
 $((FREEZE, UNFREEZE), ACTIVE),$   
 $((FREEZE, RELEASE), SEIZED),$   
 $((SEIZE, RESTRICT), ACTIVE),$   
 $((SEIZE, UNRESTRICT), RESTRICTED),$   
 $((SEIZE, RELEASE), ACTIVE),$   
 $((RESTRICT, UNFREEZE), FROZEN),$   
 $((RESTRICT, UNRESTRICT), ACTIVE),$   
 $((RESTRICT, RELEASE), SEIZED)]$

**definition** *witness-valid* ::  
 $((reg-action \times reg-action) \times reg-state) \Rightarrow bool$   
**where**  
*witness-valid*  $w =$   
 $(case\ w\ of\ ((a, b), s) \Rightarrow$   
 $state-effect\ (state-effect\ s\ a)\ b \neq$   
 $state-effect\ (state-effect\ s\ b)\ a)$

**lemma** *pair-commutes-iff*:  
 $pair-commutes\ (a, b) \longleftrightarrow commutes\ a\ b$   
**unfolding** *pair-commutes-def* *commutes-def* *all-reg-states-def*  
**by** (auto, case-tac s; auto)

**theorem** *commuting-pairs-complete*:  
 $filter\ pair-commutes\ all-unordered-pairs = commuting-pairs$   
**by** (simp add: *pair-commutes-def* *all-unordered-pairs-def* *commuting-pairs-def*  
*all-reg-states-def* *state-effect-def* *legal-outcome-def*)

**theorem** *noncommuting-pairs-complete*:  
 $filter\ (Not \circ pair-commutes)\ all-unordered-pairs = noncommuting-pairs$   
**by** (simp add: *pair-commutes-def* *all-unordered-pairs-def* *noncommuting-pairs-def*  
*all-reg-states-def* *state-effect-def* *legal-outcome-def*)

**theorem** *pair-classification-cardinality*:  
 $\text{length commuting-pairs} = 12 \wedge$   
 $\text{length noncommuting-pairs} = 9 \wedge$   
 $\text{length all-unordered-pairs} = 21$   
**by** (*simp add: commuting-pairs-def noncommuting-pairs-def all-unordered-pairs-def*)

**theorem** *noncommuting-witnesses-sound*:  
 $\text{list-all witness-valid noncommuting-witnesses}$   
**by** (*simp add: witness-valid-def noncommuting-witnesses-def state-effect-def legal-outcome-def*)

**theorem** *noncommuting-witnesses-complete*:  
 $\text{map fst noncommuting-witnesses} = \text{noncommuting-pairs}$   
**by** (*simp add: noncommuting-witnesses-def noncommuting-pairs-def*)

**lemma** *commuting-pairs-sound*:  
**assumes**  $p \in \text{set commuting-pairs}$   
**shows**  $\text{case } p \text{ of } (a, b) \Rightarrow \text{commutes } a \ b$   
**proof** –  
**have**  $p \in \text{set (filter pair-commutes all-unordered-pairs)}$   
**using** *assms commuting-pairs-complete* **by** *simp*  
**then have**  $\text{pair-commutes } p$  **by** *simp*  
**then show** *?thesis*  
**by** (*cases p*) (*simp add: pair-commutes-iff*)  
**qed**

## 4 Terminal Absorption and State Provenance

**theorem** *confiscated-word-absorbing*:  
 $\text{run-word as CONFISCATED} = \text{CONFISCATED}$   
**proof** (*induction as*)  
**case** *Nil*  
**then show** *?case* **by** *simp*  
**next**  
**case** (*Cons a as*)  
**then show** *?case*  
**by** (*cases a*) (*simp-all add: state-effect-def legal-outcome-def*)  
**qed**

**theorem** *confiscated-word-rejections*:  
 $\text{snd (action-outcome-queue as CONFISCATED)} =$   
 $\text{map } (\lambda-. \text{Rejected Undefined-Transition}) \text{ as}$   
**proof** (*induction as*)  
**case** *Nil*  
**then show** *?case* **by** *simp*  
**next**  
**case** (*Cons a as*)  
**then show** *?case*

by (cases a) (simp-all add: legal-outcome-def Let-def)  
qed

**theorem** concrete-terminal-queue-is-rejected:  
 action-outcome-queue [FREEZE] CONFISCATED =  
 (CONFISCATED, [Rejected Undefined-Transition])  
 by (simp add: legal-outcome-def)

**lemma** frozen-effect-origin:  
 state-effect s a = FROZEN  $\implies$  s = FROZEN  $\vee$  a = FREEZE  
 by (cases s; cases a; simp add: state-effect-def legal-outcome-def)

**lemma** seized-effect-origin:  
 state-effect s a = SEIZED  $\implies$  s = SEIZED  $\vee$  a = SEIZE  
 by (cases s; cases a; simp add: state-effect-def legal-outcome-def)

**lemma** confiscated-effect-origin:  
 state-effect s a = CONFISCATED  $\implies$   
 s = CONFISCATED  $\vee$  a = CONFISCATE  
 by (cases s; cases a; simp add: state-effect-def legal-outcome-def)

**theorem** concrete-provenance-witnesses:  
 state-effect ACTIVE FREEZE = FROZEN  $\wedge$   
 state-effect ACTIVE SEIZE = SEIZED  $\wedge$   
 state-effect ACTIVE CONFISCATE = CONFISCATED  
 by (simp add: state-effect-def legal-outcome-def)

**theorem** frozen-word-provenance:  
 assumes run-word as s = FROZEN and s  $\neq$  FROZEN  
 shows FREEZE  $\in$  set as  
 using assms  
**proof** (induction as arbitrary: s)  
 case Nil  
 then show ?case by simp  
**next**  
 case (Cons a as)  
 show ?case  
**proof** (cases state-effect s a = FROZEN)  
 case True  
 have a = FREEZE  
 using frozen-effect-origin[OF True] Cons.prem2 by blast  
 then show ?thesis by simp  
**next**  
 case False  
 have final: run-word as (state-effect s a) = FROZEN  
 using Cons.prem1 by simp  
 have FREEZE  $\in$  set as  
 using Cons.IH[OF final False] .  
 then show ?thesis by simp

qed  
qed

**theorem** *seized-word-provenance*:  
 assumes *run-word as s = SEIZED and s ≠ SEIZED*  
 shows *SEIZE ∈ set as*  
 using *assms*  
**proof** (*induction as arbitrary: s*)  
 case *Nil*  
 then show *?case* **by** *simp*  
**next**  
 case (*Cons a as*)  
 show *?case*  
**proof** (*cases state-effect s a = SEIZED*)  
 case *True*  
 have *a = SEIZE*  
 using *seized-effect-origin[OF True] Cons.prem(2)* **by** *blast*  
 then show *?thesis* **by** *simp*  
**next**  
 case *False*  
 have *final: run-word as (state-effect s a) = SEIZED*  
 using *Cons.prem(1)* **by** *simp*  
 have *SEIZE ∈ set as*  
 using *Cons.IH[OF final False]* .  
 then show *?thesis* **by** *simp*  
 qed  
 qed

**theorem** *confiscated-word-provenance*:  
 assumes *run-word as s = CONFISCATED and s ≠ CONFISCATED*  
 shows *CONFISCATE ∈ set as*  
 using *assms*  
**proof** (*induction as arbitrary: s*)  
 case *Nil*  
 then show *?case* **by** *simp*  
**next**  
 case (*Cons a as*)  
 show *?case*  
**proof** (*cases state-effect s a = CONFISCATED*)  
 case *True*  
 have *a = CONFISCATE*  
 using *confiscated-effect-origin[OF True] Cons.prem(2)* **by** *blast*  
 then show *?thesis* **by** *simp*  
**next**  
 case *False*  
 have *final: run-word as (state-effect s a) = CONFISCATED*  
 using *Cons.prem(1)* **by** *simp*  
 have *CONFISCATE ∈ set as*  
 using *Cons.IH[OF final False]* .

```

    then show ?thesis by simp
qed
qed

```

## 5 Typed Legal-Effect Descriptors

```

datatype legal-action-kind =
  Legal-Freeze
  | Legal-Seize
  | Legal-Confiscate
  | Legal-Restrict
  | Legal-Recover
  | Legal-Liquidate

```

```

datatype reversibility =
  Reversible
  | Conditional
  | Irreversible
  | Configurable
  | One-Time

```

```

datatype ownership-effect =
  Retained
  | Transferred
  | Terminated
  | Restored

```

```

datatype finality-effect =
  Provisional
  | Interim-Custodial
  | Final
  | Conditional-Finality
  | Restorative

```

```

record legal-effect-descriptor =
  descriptor-reversibility :: reversibility
  descriptor-ownership     :: ownership-effect
  descriptor-finality      :: finality-effect

```

```

fun legal-descriptor :: legal-action-kind  $\Rightarrow$  legal-effect-descriptor where
  legal-descriptor Legal-Freeze =
    (descriptor-reversibility = Reversible,
     descriptor-ownership = Retained,
     descriptor-finality = Provisional)
  | legal-descriptor Legal-Seize =
    (descriptor-reversibility = Conditional,
     descriptor-ownership = Retained,
     descriptor-finality = Interim-Custodial)
  | legal-descriptor Legal-Confiscate =

```

```

    (⟦descriptor-reversibility = Irreversible,
     descriptor-ownership = Transferred,
     descriptor-finality = Final⟧)
| legal-descriptor Legal-Restrict =
    (⟦descriptor-reversibility = Configurable,
     descriptor-ownership = Retained,
     descriptor-finality = Conditional-Finality⟧)
| legal-descriptor Legal-Recover =
    (⟦descriptor-reversibility = One-Time,
     descriptor-ownership = Restored,
     descriptor-finality = Restorative⟧)
| legal-descriptor Legal-Liquidate =
    (⟦descriptor-reversibility = Irreversible,
     descriptor-ownership = Terminated,
     descriptor-finality = Final⟧)

fun transition-label-of :: legal-action-kind ⇒ reg-action option where
  transition-label-of Legal-Freeze = Some FREEZE
| transition-label-of Legal-Seize = Some SEIZE
| transition-label-of Legal-Confiscate = Some CONFISCATE
| transition-label-of Legal-Restrict = Some RESTRICT
| transition-label-of Legal-Recover = None
| transition-label-of Legal-Liquidate = None

fun descriptor-target :: legal-action-kind ⇒ reg-state option where
  descriptor-target Legal-Freeze = Some FROZEN
| descriptor-target Legal-Seize = Some SEIZED
| descriptor-target Legal-Confiscate = Some CONFISCATED
| descriptor-target Legal-Restrict = Some RESTRICTED
| descriptor-target Legal-Recover = None
| descriptor-target Legal-Liquidate = None

definition all-legal-actions :: legal-action-kind list where
  all-legal-actions =
    [Legal-Freeze, Legal-Seize, Legal-Confiscate, Legal-Restrict,
     Legal-Recover, Legal-Liquidate]

theorem legal-action-descriptor-inventory:
  set all-legal-actions = UNIV ∧
  length all-legal-actions = 6 ∧
  inj legal-descriptor
proof –
  have full: set all-legal-actions = UNIV
proof (rule set-eqI)
  fix x
  show x ∈ set all-legal-actions ⟷ x ∈ UNIV
  by (cases x) (simp-all add: all-legal-actions-def)
qed
have count: length all-legal-actions = 6

```

```

    by (simp add: all-legal-actions-def)
  have injection: inj legal-descriptor
  proof (rule injI)
    fix x y
    assume legal-descriptor x = legal-descriptor y
    then show x = y by (cases x; cases y; simp-all)
  qed
  show ?thesis using full count injection by simp
  qed

```

```

theorem descriptor-transition-compatibility:
  assumes transition-label-of k = Some a
    and reg-transition s a = Some s'
  shows descriptor-target k = Some s'
  using assms
  by (cases k; cases s; auto)

```

```

theorem recover-and-liquidate-are-not-state-transitions:
  transition-label-of Legal-Recover = None ∧
  transition-label-of Legal-Liquidate = None
  by simp

```

## 6 Ordinary and Enforcement Transfer Gates

```

datatype transfer-path =
  Ordinary-Transfer
  | Authorized-Enforcement legal-action-kind

```

```

record transfer-request =
  transfer-path-value :: transfer-path
  baseline-clear      :: bool
  restriction-clear   :: bool
  enforcement-approved :: bool

```

```

fun regulatory-state-gate :: reg-state ⇒ bool ⇒ bool where
  regulatory-state-gate ACTIVE restriction = True
| regulatory-state-gate FROZEN restriction = False
| regulatory-state-gate SEIZED restriction = False
| regulatory-state-gate CONFISCATED restriction = False
| regulatory-state-gate RESTRICTED restriction = restriction

```

```

fun enforcement-transfer-action :: legal-action-kind ⇒ bool where
  enforcement-transfer-action Legal-Seize = True
| enforcement-transfer-action Legal-Confiscate = True
| enforcement-transfer-action Legal-Recover = True
| enforcement-transfer-action Legal-Liquidate = True
| enforcement-transfer-action Legal-Freeze = False
| enforcement-transfer-action Legal-Restrict = False

```

**definition** *ordinary-transfer-allowed* ::

*bool*  $\Rightarrow$  *bool*  $\Rightarrow$  *reg-state*  $\Rightarrow$  *bool*

**where**

*ordinary-transfer-allowed baseline restriction s*  $\longleftrightarrow$   
*baseline*  $\wedge$  *regulatory-state-gate s restriction*

**definition** *transfer-allowed* :: *reg-state*  $\Rightarrow$  *transfer-request*  $\Rightarrow$  *bool* **where**

*transfer-allowed s req*  $\longleftrightarrow$   
 (case *transfer-path-value req* of  
   *Ordinary-Transfer*  $\Rightarrow$   
     *ordinary-transfer-allowed*  
     (*baseline-clear req*) (*restriction-clear req*) *s*  
   | *Authorized-Enforcement k*  $\Rightarrow$   
     *enforcement-approved req*  $\wedge$  *enforcement-transfer-action k*)

**theorem** *ordinary-transfer-uses-both-gates*:

*transfer-allowed s*  
 (*transfer-path-value* = *Ordinary-Transfer*,  
  *baseline-clear* = *baseline*,  
  *restriction-clear* = *restriction*,  
  *enforcement-approved* = *authorization*)

$\longleftrightarrow$

*baseline*  $\wedge$  *regulatory-state-gate s restriction*

**by** (*simp add: transfer-allowed-def ordinary-transfer-allowed-def*)

**theorem** *restricted-transfer-uses-current-condition*:

*transfer-allowed RESTRICTED*  
 (*transfer-path-value* = *Ordinary-Transfer*,  
  *baseline-clear* = *baseline*,  
  *restriction-clear* = *restriction*,  
  *enforcement-approved* = *authorization*)

$\longleftrightarrow$  *baseline*  $\wedge$  *restriction*

**by** (*simp add: transfer-allowed-def ordinary-transfer-allowed-def*)

**theorem** *enforcement-transfer-is-a-separate-path*:

*transfer-allowed s*  
 (*transfer-path-value* = *Authorized-Enforcement k*,  
  *baseline-clear* = *baseline*,  
  *restriction-clear* = *restriction*,  
  *enforcement-approved* = *authorization*)

$\longleftrightarrow$  *authorization*  $\wedge$  *enforcement-transfer-action k*

**by** (*simp add: transfer-allowed-def*)

**definition** *concrete-ordinary-request* :: *transfer-request* **where**

*concrete-ordinary-request* =  
 (*transfer-path-value* = *Ordinary-Transfer*,  
  *baseline-clear* = *True*,  
  *restriction-clear* = *True*,  
  *enforcement-approved* = *False*)

**definition** *concrete-restricted-request-blocked* :: *transfer-request* **where**  
*concrete-restricted-request-blocked* =  
*concrete-ordinary-request*(*restriction-clear* := *False*)

**definition** *concrete-enforcement-request* :: *transfer-request* **where**  
*concrete-enforcement-request* =  
(*transfer-path-value* = *Authorized-Enforcement Legal-Recover*,  
*baseline-clear* = *False*,  
*restriction-clear* = *False*,  
*enforcement-approved* = *True*)

**theorem** *concrete-gate-premises-activate*:  
*transfer-allowed ACTIVE concrete-ordinary-request*  $\wedge$   
*transfer-allowed RESTRICTED concrete-ordinary-request*  $\wedge$   
 $\neg$  *transfer-allowed RESTRICTED concrete-restricted-request-blocked*  $\wedge$   
 $\neg$  *transfer-allowed FROZEN concrete-ordinary-request*  $\wedge$   
*transfer-allowed FROZEN concrete-enforcement-request*  
**by** (*simp add: concrete-ordinary-request-def*  
*concrete-restricted-request-blocked-def concrete-enforcement-request-def*  
*transfer-allowed-def ordinary-transfer-allowed-def*)

## 7 Reference Roundtrips and Information Loss

**theorem** *freeze-roundtrip*:  
*run-word* [*FREEZE*, *UNFREEZE*] *ACTIVE* = *ACTIVE*  
**by** (*simp add: state-effect-def legal-outcome-def*)

**theorem** *seizure-roundtrip*:  
*run-word* [*SEIZE*, *RELEASE*] *ACTIVE* = *ACTIVE*  
**by** (*simp add: state-effect-def legal-outcome-def*)

**theorem** *restriction-roundtrip*:  
*run-word* [*RESTRICT*, *UNRESTRICT*] *ACTIVE* = *ACTIVE*  
**by** (*simp add: state-effect-def legal-outcome-def*)

**theorem** *restricted-freeze-unfreeze-loses-history*:  
*run-word* [*FREEZE*, *UNFREEZE*] *RESTRICTED* = *ACTIVE*  $\wedge$   
*run-word* [*FREEZE*, *UNFREEZE*] *RESTRICTED*  $\neq$  *RESTRICTED*  
**by** (*simp add: state-effect-def legal-outcome-def*)

## 8 Commands, Trace, and Frame Properties

**datatype** *execution-status* =  
*Ready*  
| *Failed operational-failure-reason*

**record** *regulatory-command* =

```

command-identity :: nat
command-asset    :: asset-id
command-actor    :: nat
command-source   :: chain-id
command-action   :: reg-action
command-status   :: execution-status

```

```

record trace-event =
  event-identity      :: nat
  event-asset         :: asset-id
  event-actor         :: nat
  event-source        :: chain-id
  event-action        :: reg-action
  event-previous-state :: reg-state option
  event-outcome       :: command-outcome

```

**definition** *event-for* ::

*regulatory-command*  $\Rightarrow$  *reg-state option*  $\Rightarrow$  *command-outcome*  $\Rightarrow$  *trace-event*

**where**

```

event-for cmd previous out =
  (event-identity = command-identity cmd,
   event-asset = command-asset cmd,
   event-actor = command-actor cmd,
   event-source = command-source cmd,
   event-action = command-action cmd,
   event-previous-state = previous,
   event-outcome = out)

```

**definition** *execute-command* ::

*reg-state*  $\Rightarrow$  *regulatory-command*  $\Rightarrow$  *reg-state*  $\times$  *trace-event*

**where**

```

execute-command s cmd =
  (case command-status cmd of
    Failed reason  $\Rightarrow$ 
      (s, event-for cmd (Some s) (Operational-Failure reason))
  | Ready  $\Rightarrow$ 
      (let out = legal-outcome s (command-action cmd)
       in (state-after s out, event-for cmd (Some s) out)))

```

**fun** *run-command-queue* ::

*regulatory-command list*  $\Rightarrow$  *reg-state*  $\Rightarrow$  *reg-state*  $\times$  *trace-event list*

**where**

```

run-command-queue [] s = (s, [])
| run-command-queue (cmd # cmds) s =
  (let first = execute-command s cmd;
   rest = run-command-queue cmds (fst first)
   in (fst rest, snd first # snd rest))

```

**theorem** *command-queue-preserves-order-and-length:*  
 $\text{map event-identity (snd (run-command-queue cmds s))} =$   
 $\text{map command-identity cmds} \wedge$   
 $\text{length (snd (run-command-queue cmds s))} = \text{length cmds}$   
**proof** (*induction cmds arbitrary: s*)  
 case Nil  
 then show ?case by simp  
next  
 case (Cons cmd cmds)  
 then show ?case  
 by (cases command-status cmd)  
 (simp-all add: execute-command-def event-for-def Let-def)  
qed

**theorem** *command-trace-preserves-input:*  
 $\text{event-identity (snd (execute-command s cmd))} = \text{command-identity cmd} \wedge$   
 $\text{event-asset (snd (execute-command s cmd))} = \text{command-asset cmd} \wedge$   
 $\text{event-actor (snd (execute-command s cmd))} = \text{command-actor cmd} \wedge$   
 $\text{event-source (snd (execute-command s cmd))} = \text{command-source cmd} \wedge$   
 $\text{event-action (snd (execute-command s cmd))} = \text{command-action cmd}$   
**by** (cases command-status cmd)  
 (simp-all add: execute-command-def event-for-def Let-def)

**theorem** *command-trace-records-execution-outcome:*  
 $\text{event-outcome (snd (execute-command s cmd))} =$   
 (case command-status cmd of  
 Failed reason  $\Rightarrow$  Operational-Failure reason  
 | Ready  $\Rightarrow$  legal-outcome s (command-action cmd))  
**by** (cases command-status cmd)  
 (simp-all add: execute-command-def event-for-def Let-def)

**type-synonym** *asset-store* = *asset-id*  $\Rightarrow$  *reg-state*

**definition** *execute-on-store* ::  
*regulatory-command*  $\Rightarrow$  *asset-store*  $\Rightarrow$  *asset-store*  $\times$  *trace-event*  
**where**

*execute-on-store cmd store* =  
 (let result = execute-command (store (command-asset cmd)) cmd  
 in (store (command-asset cmd := fst result), snd result))

**theorem** *other-asset-unchanged:*  
**assumes** *other*  $\neq$  *command-asset cmd*  
**shows** *fst (execute-on-store cmd store)* *other* = *store other*  
**using** *assms* **by** (*simp add: execute-on-store-def Let-def*)

**definition** *concrete-message* :: *regulatory-command* **where**  
*concrete-message* =  
 (command-identity = 1001,  
 command-asset = 7,

$command-actor = 42,$   
 $command-source = 3,$   
 $command-action = FREEZE,$   
 $command-status = Ready)$

**definition**  $repeated-message :: regulatory-command$  **where**  
 $repeated-message = concrete-message(\ command-identity := 1002)$

**theorem**  $concrete-message-premises-activate:$   
 $command-status\ concrete-message = Ready \wedge$   
 $reg-transition\ ACTIVE\ (command-action\ concrete-message) = Some\ FROZEN$   
 $\wedge$   
 $execute-command\ ACTIVE\ concrete-message =$   
 $(FROZEN, event-for\ concrete-message\ (Some\ ACTIVE)\ (Applied\ FROZEN))$   
**by** ( $simp\ add: concrete-message-def\ execute-command-def\ legal-outcome-def$   
 $Let-def$ )

**theorem**  $repeated-message-has-idempotent-state-and-distinct-trace:$   
 $fst\ (execute-command\ FROZEN\ repeated-message) = FROZEN \wedge$   
 $event-outcome\ (snd\ (execute-command\ FROZEN\ repeated-message)) =$   
 $Rejected\ Undefined-Transition \wedge$   
 $snd\ (execute-command\ ACTIVE\ concrete-message) \neq$   
 $snd\ (execute-command\ FROZEN\ repeated-message)$   
**by** ( $simp\ add: concrete-message-def\ repeated-message-def\ execute-command-def$   
 $event-for-def\ legal-outcome-def\ Let-def$ )

## 9 Atomic Synchronization Queues

**definition**  $execute-sync-command ::$   
 $regulatory-command \Rightarrow global-state \Rightarrow global-state \times trace-event$   
**where**  
 $execute-sync-command\ cmd\ gs =$   
 $(case\ command-status\ cmd\ of$   
 $\quad Failed\ reason \Rightarrow$   
 $\quad (gs, event-for\ cmd$   
 $\quad (get-reg-state\ gs\ (command-source\ cmd)\ (command-asset\ cmd))$   
 $\quad (Operational-Failure\ reason))$   
 $\mid Ready \Rightarrow$   
 $\quad (case\ get-reg-state\ gs\ (command-source\ cmd)\ (command-asset\ cmd)\ of$   
 $\quad \quad None \Rightarrow$   
 $\quad \quad (gs, event-for\ cmd\ None\ (Operational-Failure\ Missing-Asset))$   
 $\quad \mid Some\ s \Rightarrow$   
 $\quad \quad (case\ reg-transition\ s\ (command-action\ cmd)\ of$   
 $\quad \quad \quad None \Rightarrow$   
 $\quad \quad \quad (gs, event-for\ cmd\ (Some\ s)$   
 $\quad \quad \quad (Rejected\ Undefined-Transition))$   
 $\quad \quad \mid Some\ s' \Rightarrow$   
 $\quad \quad \quad (case\ sync\ (command-source\ cmd)\ (command-action\ cmd)$   
 $\quad \quad \quad (command-asset\ cmd)\ gs\ of$

$$\begin{aligned}
& \text{None} \Rightarrow \\
& \quad (gs, \text{event-for cmd (Some s)} \\
& \quad \quad (\text{Operational-Failure Lock-Unavailable})) \\
& | \text{Some gs'} \Rightarrow \\
& \quad (gs', \text{event-for cmd (Some s) (Applied s')})
\end{aligned}$$

```

lemma sync-command-preserves-valid-state:
  assumes valid-state gs
  shows valid-state (fst (execute-sync-command cmd gs))
proof (cases command-status cmd)
  case (Failed reason)
  then show ?thesis
    using assms by (simp add: execute-sync-command-def)
next
  case Ready
  show ?thesis
  proof (cases get-reg-state gs (command-source cmd) (command-asset cmd))
    case None
    note current = None
    with Ready assms show ?thesis
    by (simp add: execute-sync-command-def)
  next
    case (Some s)
    note current = Some
    show ?thesis
    proof (cases reg-transition s (command-action cmd))
      case None
      note transition = None
      with Ready current assms show ?thesis
      by (simp add: execute-sync-command-def)
    next
      case (Some s')
      note transition = Some
      show ?thesis
      proof (cases sync (command-source cmd) (command-action cmd)
        (command-asset cmd) gs)
        case None
        note synced = None
        with Ready current transition assms show ?thesis
        by (simp add: execute-sync-command-def)
      next
        case (Some gs')
        note synced = Some
        have valid-state gs'
          using valid-state-preservation[OF assms current transition synced] .
        with Ready current transition synced show ?thesis
        by (simp add: execute-sync-command-def)
      qed
    qed

```

```

qed
qed

fun run-sync-queue ::
  regulatory-command list  $\Rightarrow$  global-state  $\Rightarrow$  global-state  $\times$  trace-event list
where
  run-sync-queue [] gs = (gs, [])
| run-sync-queue (cmd # cmds) gs =
  (let first = execute-sync-command cmd gs;
   rest = run-sync-queue cmds (fst first)
   in (fst rest, snd first # snd rest))

theorem sync-queue-preserves-valid-state:
  assumes valid-state gs
  shows valid-state (fst (run-sync-queue cmds gs))
  using assms
proof (induction cmds arbitrary: gs)
  case Nil
  then show ?case by simp
next
  case (Cons cmd cmds)
  have first: valid-state (fst (execute-sync-command cmd gs))
    using sync-command-preserves-valid-state[OF Cons.premis] .
  have rest:
    valid-state (fst (run-sync-queue cmds
      (fst (execute-sync-command cmd gs))))
    using Cons.IH[OF first] .
  then show ?case by (simp add: Let-def)
qed

fun completed-states ::
  regulatory-command list  $\Rightarrow$  global-state  $\Rightarrow$  global-state list
where
  completed-states [] gs = [gs]
| completed-states (cmd # cmds) gs =
  gs # completed-states cmds (fst (execute-sync-command cmd gs))

theorem completed-states-are-valid:
  assumes valid-state gs
  shows list-all valid-state (completed-states cmds gs)
  using assms
proof (induction cmds arbitrary: gs)
  case Nil
  then show ?case by simp
next
  case (Cons cmd cmds)
  have next-valid: valid-state (fst (execute-sync-command cmd gs))
    using sync-command-preserves-valid-state[OF Cons.premis] .
  then show ?case using Cons.IH Cons.premis by simp

```

**qed**

**theorem** *completed-prefixes-are-consistent:*

**assumes** *valid-state gs*

**shows** *list-all consistent-state (completed-states cmds gs)*

**using** *completed-states-are-valid[OF assms]*

**unfolding** *valid-state-def list-all-iff* **by** *blast*

**definition** *concrete-global-state :: global-state where*

*concrete-global-state =*

*(\gs-chains = (\cid aid.*

*if cid = 0 ∧ aid = 0 then Some (\as-reg-state = ACTIVE)*

*else None),*

*gs-locks = (\-. False))*

**definition** *concrete-sync-command :: regulatory-command where*

*concrete-sync-command =*

*(\command-identity = 2001,*

*command-asset = 0,*

*command-actor = 9,*

*command-source = 0,*

*command-action = FREEZE,*

*command-status = Ready)*

**lemma** *concrete-global-state-get-reg:*

*get-reg-state concrete-global-state cid aid =*

*(if cid = 0 ∧ aid = 0 then Some ACTIVE else None)*

**by** (*simp add: concrete-global-state-def get-reg-state-def get-asset-state-def*  
*split: if-splits*)

**lemma** *concrete-global-state-valid:*

*valid-state concrete-global-state*

**proof** —

**have** *consistent-state concrete-global-state*

**unfolding** *consistent-state-def*

**by** (*auto simp: concrete-global-state-get-reg split: if-splits*)

**moreover have** *no-locked-without-reason concrete-global-state*

**by** (*simp add: no-locked-without-reason-def is-locked-def*

*concrete-global-state-def*)

**ultimately show** *?thesis* **by** (*simp add: valid-state-def*)

**qed**

**theorem** *concrete-sync-premises-activate:*

*valid-state concrete-global-state ∧*

*get-reg-state concrete-global-state 0 0 = Some ACTIVE ∧*

*reg-transition ACTIVE FREEZE = Some FROZEN ∧*

*event-outcome (snd (execute-sync-command concrete-sync-command*  
*concrete-global-state)) = Applied FROZEN ∧*

*get-reg-state (fst (execute-sync-command concrete-sync-command*

```

    concrete-global-state)) 0 0 = Some FROZEN
proof –
  have valid: valid-state concrete-global-state
  using concrete-global-state-valid .
  have current:
    get-reg-state concrete-global-state 0 0 = Some ACTIVE
  by (simp add: concrete-global-state-get-reg)
  have applied:
    event-outcome (snd (execute-sync-command concrete-sync-command
      concrete-global-state)) = Applied FROZEN
  by (simp add: concrete-global-state-def concrete-sync-command-def
    execute-sync-command-def sync-def get-reg-state-def get-asset-state-def
    acquire-lock-def is-locked-def connected-chains-def asset-exists-def
    update-all-chains-def release-lock-def event-for-def Let-def)
  have final:
    get-reg-state (fst (execute-sync-command concrete-sync-command
      concrete-global-state)) 0 0 = Some FROZEN
  by (simp add: concrete-global-state-def concrete-sync-command-def
    execute-sync-command-def sync-def get-reg-state-def get-asset-state-def
    acquire-lock-def is-locked-def connected-chains-def asset-exists-def
    update-all-chains-def release-lock-def event-for-def Let-def)
  show ?thesis using valid current applied final by simp
qed

```

## 10 Finite Transformation Normal Forms

**definition** *state-vector* :: *reg-action list*  $\Rightarrow$  *reg-state list* **where**  
*state-vector as* = *map (run-word as) all-reg-states*

**definition** *vector-step* ::  
*reg-state list*  $\Rightarrow$  *reg-action*  $\Rightarrow$  *reg-state list*  
**where**  
*vector-step v a* = *map ( $\lambda s$ . state-effect s a) v*

**definition** *normal-forms* :: *reg-state list list* **where**  
*normal-forms* =  
 [[ACTIVE, FROZEN, SEIZED, CONFISCATED, RESTRICTED],  
 [CONFISCATED, CONFISCATED, CONFISCATED, CONFISCATED, CONFISCATED],  
 [FROZEN, FROZEN, SEIZED, CONFISCATED, FROZEN],  
 [ACTIVE, FROZEN, ACTIVE, CONFISCATED, RESTRICTED],  
 [RESTRICTED, FROZEN, SEIZED, CONFISCATED, RESTRICTED],  
 [SEIZED, SEIZED, SEIZED, CONFISCATED, RESTRICTED],  
 [ACTIVE, ACTIVE, SEIZED, CONFISCATED, RESTRICTED],  
 [ACTIVE, FROZEN, SEIZED, CONFISCATED, ACTIVE],  
 [FROZEN, FROZEN, ACTIVE, CONFISCATED, FROZEN],  
 [SEIZED, SEIZED, SEIZED, CONFISCATED, SEIZED],  
 [ACTIVE, ACTIVE, SEIZED, CONFISCATED, ACTIVE],  
 [FROZEN, FROZEN, FROZEN, CONFISCATED, FROZEN],

[RESTRICTED, FROZEN, RESTRICTED, CONFISCATED, RE-  
 STRICTED],  
 [RESTRICTED, FROZEN, ACTIVE, CONFISCATED, RESTRICTED],  
 [RESTRICTED, SEIZED, SEIZED, CONFISCATED, RESTRICTED],  
 [RESTRICTED, ACTIVE, SEIZED, CONFISCATED, RESTRICTED],  
 [SEIZED, SEIZED, SEIZED, CONFISCATED, FROZEN],  
 [ACTIVE, ACTIVE, ACTIVE, CONFISCATED, RESTRICTED],  
 [SEIZED, SEIZED, SEIZED, CONFISCATED, ACTIVE],  
 [RESTRICTED, RESTRICTED, SEIZED, CONFISCATED, RESTRICTED],  
 [ACTIVE, FROZEN, ACTIVE, CONFISCATED, ACTIVE],  
 [FROZEN, FROZEN, RESTRICTED, CONFISCATED, FROZEN],  
 [ACTIVE, ACTIVE, ACTIVE, CONFISCATED, ACTIVE],  
 [RESTRICTED, SEIZED, RESTRICTED, CONFISCATED, RESTRICTED],  
 [RESTRICTED, ACTIVE, RESTRICTED, CONFISCATED, RESTRICTED],  
 [FROZEN, SEIZED, SEIZED, CONFISCATED, FROZEN],  
 [RESTRICTED, ACTIVE, ACTIVE, CONFISCATED, RESTRICTED],  
 [ACTIVE, SEIZED, SEIZED, CONFISCATED, ACTIVE],  
 [ACTIVE, ACTIVE, ACTIVE, CONFISCATED, FROZEN],  
 [RESTRICTED, RESTRICTED, RESTRICTED, CONFISCATED, RE-  
 STRICTED],  
 [RESTRICTED, RESTRICTED, ACTIVE, CONFISCATED, RESTRICTED],  
 [SEIZED, SEIZED, RESTRICTED, CONFISCATED, SEIZED],  
 [ACTIVE, ACTIVE, RESTRICTED, CONFISCATED, ACTIVE],  
 [FROZEN, SEIZED, FROZEN, CONFISCATED, FROZEN],  
 [ACTIVE, SEIZED, ACTIVE, CONFISCATED, ACTIVE],  
 [FROZEN, ACTIVE, ACTIVE, CONFISCATED, FROZEN],  
 [RESTRICTED, RESTRICTED, RESTRICTED, CONFISCATED,  
 FROZEN],  
 [SEIZED, SEIZED, FROZEN, CONFISCATED, SEIZED],  
 [SEIZED, SEIZED, ACTIVE, CONFISCATED, SEIZED],  
 [FROZEN, ACTIVE, FROZEN, CONFISCATED, FROZEN],  
 [FROZEN, RESTRICTED, RESTRICTED, CONFISCATED, FROZEN],  
 [RESTRICTED, RESTRICTED, RESTRICTED, CONFISCATED, SEIZED],  
 [RESTRICTED, RESTRICTED, RESTRICTED, CONFISCATED, ACTIVE],  
 [ACTIVE, ACTIVE, FROZEN, CONFISCATED, ACTIVE],  
 [FROZEN, RESTRICTED, FROZEN, CONFISCATED, FROZEN],  
 [SEIZED, RESTRICTED, RESTRICTED, CONFISCATED, SEIZED],  
 [ACTIVE, RESTRICTED, RESTRICTED, CONFISCATED, ACTIVE],  
 [FROZEN, FROZEN, FROZEN, CONFISCATED, SEIZED],  
 [ACTIVE, ACTIVE, ACTIVE, CONFISCATED, SEIZED],  
 [RESTRICTED, RESTRICTED, FROZEN, CONFISCATED, RE-  
 STRICTED],  
 [SEIZED, RESTRICTED, SEIZED, CONFISCATED, SEIZED],  
 [ACTIVE, RESTRICTED, ACTIVE, CONFISCATED, ACTIVE],  
 [SEIZED, FROZEN, FROZEN, CONFISCATED, SEIZED],  
 [SEIZED, ACTIVE, ACTIVE, CONFISCATED, SEIZED],  
 [FROZEN, FROZEN, FROZEN, CONFISCATED, ACTIVE],  
 [SEIZED, FROZEN, SEIZED, CONFISCATED, SEIZED],  
 [SEIZED, ACTIVE, SEIZED, CONFISCATED, SEIZED],

[ACTIVE, FROZEN, FROZEN, CONFISCATED, ACTIVE],  
[FROZEN, FROZEN, FROZEN, CONFISCATED, RESTRICTED],  
[RESTRICTED, FROZEN, FROZEN, CONFISCATED, RESTRICTED]]

**definition** *normal-form-words* :: *reg-action list list* **where**

*normal-form-words* =

[[],  
[CONFISCATE],  
[FREEZE],  
[RELEASE],  
[RESTRICT],  
[SEIZE],  
[UNFREEZE],  
[UNRESTRICT],  
[FREEZE, RELEASE],  
[FREEZE, SEIZE],  
[FREEZE, UNFREEZE],  
[RELEASE, FREEZE],  
[RELEASE, RESTRICT],  
[RESTRICT, RELEASE],  
[RESTRICT, SEIZE],  
[RESTRICT, UNFREEZE],  
[SEIZE, FREEZE],  
[SEIZE, RELEASE],  
[SEIZE, UNRESTRICT],  
[UNFREEZE, RESTRICT],  
[UNRESTRICT, RELEASE],  
[FREEZE, RELEASE, RESTRICT],  
[FREEZE, SEIZE, RELEASE],  
[RELEASE, RESTRICT, SEIZE],  
[RELEASE, RESTRICT, UNFREEZE],  
[RESTRICT, SEIZE, FREEZE],  
[RESTRICT, SEIZE, RELEASE],  
[RESTRICT, SEIZE, UNRESTRICT],  
[SEIZE, FREEZE, RELEASE],  
[SEIZE, RELEASE, RESTRICT],  
[UNFREEZE, RESTRICT, RELEASE],  
[FREEZE, RELEASE, RESTRICT, SEIZE],  
[FREEZE, RELEASE, RESTRICT, UNFREEZE],  
[RELEASE, RESTRICT, SEIZE, FREEZE],  
[RELEASE, RESTRICT, SEIZE, UNRESTRICT],  
[RESTRICT, SEIZE, FREEZE, RELEASE],  
[SEIZE, FREEZE, RELEASE, RESTRICT],  
[FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE],  
[FREEZE, RELEASE, RESTRICT, SEIZE, UNRESTRICT],  
[RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE],  
[RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT],  
[SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE],  
[SEIZE, FREEZE, RELEASE, RESTRICT, UNFREEZE],

[FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE],  
 [RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT],  
 [RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE],  
 [RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, UNFREEZE],  
 [SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE],  
 [SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, UNRESTRICT],  
 [FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE,  
 RESTRICT],  
 [RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE],  
 [RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, UN-  
 FREEZE],  
 [RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE],  
 [RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, UNRE-  
 STRICT],  
 [SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE],  
 [RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE,  
 FREEZE],  
 [RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE,  
 UNRESTRICT],  
 [RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE,  
 RELEASE],  
 [SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE, RELEASE,  
 RESTRICT],  
 [RESTRICT, SEIZE, FREEZE, RELEASE, RESTRICT, SEIZE, FREEZE,  
 RELEASE,  
 RESTRICT]]

**lemma** *normal-form-words-evaluate:*

*map state-vector normal-form-words = normal-forms*

**by** (*simp add: normal-form-words-def normal-forms-def state-vector-def  
 all-reg-states-def state-effect-def legal-outcome-def*)

**theorem** *normal-forms-reachable:*

**assumes** *v ∈ set normal-forms*

**shows**  $\exists as \in \text{set normal-form-words. state-vector } as = v$

**proof** –

**have** *v ∈ set (map state-vector normal-form-words)*

**using** *assms* **by** (*simp add: normal-form-words-evaluate*)

**then show** *?thesis* **by** *auto*

**qed**

**theorem** *normal-forms-cardinality:*

*length normal-forms = 60*

**by** (*simp add: normal-forms-def*)

**lemma** *normal-forms-closed-freeze:*

**assumes** *v ∈ set normal-forms*

**shows** *vector-step v FREEZE ∈ set normal-forms*

**using** *assms* **by** (*auto simp: normal-forms-def vector-step-def*)

*state-effect-def legal-outcome-def*)

**lemma** *normal-forms-closed-seize:*

**assumes**  $v \in \text{set normal-forms}$

**shows**  $\text{vector-step } v \text{ SEIZE} \in \text{set normal-forms}$

**using** *assms by* (*auto simp: normal-forms-def vector-step-def*  
*state-effect-def legal-outcome-def*)

**lemma** *normal-forms-closed-confiscate:*

**assumes**  $v \in \text{set normal-forms}$

**shows**  $\text{vector-step } v \text{ CONFISCATE} \in \text{set normal-forms}$

**using** *assms by* (*auto simp: normal-forms-def vector-step-def*  
*state-effect-def legal-outcome-def*)

**lemma** *normal-forms-closed-restrict:*

**assumes**  $v \in \text{set normal-forms}$

**shows**  $\text{vector-step } v \text{ RESTRICT} \in \text{set normal-forms}$

**using** *assms by* (*auto simp: normal-forms-def vector-step-def*  
*state-effect-def legal-outcome-def*)

**lemma** *normal-forms-closed-unfreeze:*

**assumes**  $v \in \text{set normal-forms}$

**shows**  $\text{vector-step } v \text{ UNFREEZE} \in \text{set normal-forms}$

**using** *assms by* (*auto simp: normal-forms-def vector-step-def*  
*state-effect-def legal-outcome-def*)

**lemma** *normal-forms-closed-unrestrict:*

**assumes**  $v \in \text{set normal-forms}$

**shows**  $\text{vector-step } v \text{ UNRESTRICT} \in \text{set normal-forms}$

**using** *assms by* (*auto simp: normal-forms-def vector-step-def*  
*state-effect-def legal-outcome-def*)

**lemma** *normal-forms-closed-release:*

**assumes**  $v \in \text{set normal-forms}$

**shows**  $\text{vector-step } v \text{ RELEASE} \in \text{set normal-forms}$

**using** *assms by* (*auto simp: normal-forms-def vector-step-def*  
*state-effect-def legal-outcome-def*)

**theorem** *normal-forms-closed:*

**assumes**  $v \in \text{set normal-forms}$

**shows**  $\text{vector-step } v \text{ } a \in \text{set normal-forms}$

**using** *assms*

**by** (*cases a*)

(*simp-all add: normal-forms-closed-freeze normal-forms-closed-seize*  
*normal-forms-closed-confiscate normal-forms-closed-restrict*  
*normal-forms-closed-unfreeze normal-forms-closed-unrestrict*  
*normal-forms-closed-release*)

**lemma** *state-vector-append:*

```

    state-vector (as @ [a]) = vector-step (state-vector as) a
  unfolding state-vector-def vector-step-def
  by (simp add: run-word-append all-reg-states-def)

theorem every-word-has-normal-form:
  state-vector as ∈ set normal-forms
proof (induction as rule: rev-induct)
  case Nil
  then show ?case
    unfolding state-vector-def normal-forms-def all-reg-states-def by simp
next
  case (snoc a as)
  have vector-step (state-vector as) a ∈ set normal-forms
    using normal-forms-closed[OF snoc.IH] .
  then show ?case using state-vector-append by simp
qed

theorem normal-forms-are-distinct:
  distinct normal-forms
  by (simp add: normal-forms-def)

end

```

## 11 Conclusion

The checked algebra shows where regulatory actions are order-insensitive and where order is semantically load-bearing. It also supplies a bounded, auditable normal-form universe for regression tests and later implementation mapping, without extending the claims beyond the concrete reference machine.

## References

- [1] Jinwook Kim. The cross-domain state preservation functor: A mechanized theory of regulatory state synchronization in isabelle/hol, 2026.
- [2] Jinwook Kim and Jonghun Hong. A regulatory compliance protocol for asset interoperability between traditional and decentralized finance in tokenized capital markets, 2026.